Laboratorio 12

Sarah Rachel Estrada Banilla

Problema 1:

- Escriba la reducción-β de la operación lógica NOT

1. evaluamos: $(\lambda b.b\,F\,T)\,T$.

   expresión inicial: $(\lambda b.b\,F\,T)(\lambda x.\lambda y.x)$

   primera reducción β: $(\lambda x.\lambda y.x)\,F\,T$ — al sustituir b por $(\lambda x.\lambda y.x)$.

   segunda reducción β: $(\lambda y.F)\,T$ — al aplicar T y sustituir x por F

   tercera reducción β: F — sustituimos y por T

2. evaluamos: $(\lambda b.b\,F\,T)\,F$.

   expresión inicial: $(\lambda b.b\,F\,T)(\lambda x.\lambda y.y)$

   primera reducción β: $(\lambda x.\lambda y.y)\,F\,T$ — al sustituir b por $(\lambda x.\lambda y.x)$.

   segunda reducción β: $(\lambda y.y)\,T$ — al aplicar T y sustituir x por F

   tercera reducción β: T — sustituimos y por T

- Escriba y explique como se vería la recursión y ciclos.

Para la recursión deberíamos tener una función intermedia F que recibe como primer argumento a "sí misma". y como segundo argumento el dato a procesar (n).

$F = \lambda rec.\lambda n.$ (SI $n == 0$ ENTONCES 1 SINO $n \times (rec\,(n-1))$)

Si aplicamos el combinador Y a esta función F, obtenemos la función recursiva:

Factorial $= Y\,F$

Entonces tenemos una expresión inicial: $(Y\,F)\,n$

propiedad de Y: $(F\,(Y\,F))\,n$ por reducción β

sustitución en F: $\lambda n.$ (SI $n == 0$ ENTONCES 1 SINO $n \times (rec\,(n-1))$)

paso recursivo: si la condición del "SINO" se cumple vuelve a llamar a (Y F), lo que expande una nueva copia de la función para el caso n-1.

Para un ciclo, en la programación funcional, los ciclos son simplemente funciones recursivas que mantienen un "estado" que cambia en cada iteración hasta que se cumple una condición de parada como: while(condición) { algo}. Por lo que se pasa el estado inicial, se coloca una condición de parada (SI condicional TF). Si el Verdadero, ejecuta la lógica y

Created by Notein

se llama a rec con el nuevo estado modificado. Si es falso, se devuelve el resultado acumulado y se detiene la expansión de Y.

- Explique cuando es prudente usar este tipo de programación y cuando no. De un ejemplo para cada caso.

  - Si es prudente si queremos transformar colecciones y construir cálculos como composición de funciones. Por ejemplo: una lista de ventas, de esta podemos filtrar ventas aprobadas, obtener impuestos y sumarlos. Cada paso expresa una transformación concreta y evita modificar la lista original.

  - No es prudente cuando existen recorridos muy largos, pues la recursión puede agotar la pila. Crear muchas colecciones intermedias también pueden tener un costo muy alto en la memoria. Por ejemplo: sumar un millón de enteros, en ciertos lenguajes puede superar el límite de llamadas, o un cálculo sobre una base de datos con millones de datos.

Created by Notein